Python Activity-Based Learning

Complete Solutions for Case Studies 11–20

Each case study includes Problem Statement, Objectives, Assumptions, Algorithm, Pseudocode, Flowchart, Python Source Code, Test Cases, Sample Output, and Conclusion.

Note: Flowcharts are provided as clear text-arrow sequences so they can be copied into a hand-drawn or digital flowchart.

11. Examination Result and Grace-Mark Validation

1. Problem Statement

Write a Python program to process a student's marks in multiple subjects, check subject-wise pass criteria, apply a grace-mark policy when permitted, and print the final result without allowing any mark to exceed the maximum.

2. Objectives

• Read marks for each subject.
• Check pass marks subject by subject.
• Apply grace marks only to a subject just below the pass mark.
• Ensure final marks do not exceed the maximum.
• Display marks and final pass/fail status.

3. Assumptions

• There are three subjects, each out of 100.
• The pass mark is 40 in each subject.
• At most 5 grace marks may be added to one subject if the original mark is 35–39.
• Final marks cannot exceed 100.

4. Algorithm

1. Start.
2. Store subject marks and set pass mark = 40.
3. For each subject, if the mark is from 35 to 39, add enough grace marks to reach 40, but no more than 5.
4. Limit every final mark to 100.
5. If all final marks are at least 40, display PASS; otherwise display FAIL.
6. Stop.

5. Pseudocode

```
START
marks = [38, 67, 81]
FOR each mark: IF 35 <= mark < 40, set mark = 40
Check that each mark is between 0 and 100
IF every mark >= 40, print PASS ELSE print FAIL
STOP
```

6. Flowchart

START → Enter marks → Check each subject → Apply allowed grace marks → Validate maximum marks → All subjects pass? → YES: PASS / NO: FAIL → Display final marks → STOP

7. Python Source Code

```
marks = [38, 67, 81]
pass_mark = 40
maximum_mark = 100
final_marks = []

for mark in marks:
    if mark < 0 or mark > maximum_mark:
        print("Invalid mark:", mark)
        break
    if 35 <= mark < pass_mark:
        mark = min(pass_mark, mark + 5)
    final_marks.append(mark)

if len(final_marks) == len(marks):
    print("Final marks:", final_marks)
    if all(mark >= pass_mark for mark in final_marks):
        print("Result: PASS")
    else:
        print("Result: FAIL")
```

8. Test Cases

Test Input	Expected / Checked Result
38, 67, 81	[40, 67, 81]; PASS
32, 67, 81	[32, 67, 81]; FAIL

Test Input	Expected / Checked Result
45, 56, 70	[45, 56, 70]; PASS

Sample Output

```
Final marks: [40, 67, 81]\nResult: PASS
```

9. Conclusion

The program checks subject-wise pass marks, applies the stated grace policy, and reports the final result while keeping marks within the permitted range.

12. Attendance Recovery Computation

1. Problem Statement

Develop a program that calculates current attendance and determines how many consecutive classes a student must attend to reach a required attendance percentage.

2. Objectives

• Calculate attendance percentage.
• Compare attendance with the required threshold.
• Find the minimum consecutive classes needed to reach the threshold.
• Display the result clearly.

3. Assumptions

• Attended classes = 36 and conducted classes = 50 in the sample.
• The required attendance is 75%.
• The student attends every upcoming class.

4. Algorithm

1. Start.
2. Read attended classes A, conducted classes T, and required percentage R.
3. Calculate current percentage as A/T × 100.
4. If current percentage is at least R, print eligible and zero extra classes.
5. Otherwise increase future classes one by one until A+n divided by T+n reaches R.
6. Display the minimum n and stop.

5. Pseudocode

```
START
Read A, T, R
P = A / T * 100
IF P >= R, print 0 extra classes
ELSE set n = 0; WHILE (A+n)/(T+n)*100 < R: n = n+1
Print P and n
STOP
```

6. Flowchart

START → Input attended and conducted classes → Calculate percentage → Already eligible? → YES: 0 classes / NO: count future attended classes until threshold → Display result → STOP

7. Python Source Code

```
attended = 36
conducted = 50
required = 75

if conducted <= 0 or attended < 0 or attended > conducted:
    print("Invalid attendance data")
else:
    percentage = attended / conducted * 100
    extra = 0
    while (attended + extra) / (conducted + extra) * 100 < required:
        extra += 1
    print("Current attendance:", round(percentage, 2), "%")
    print("Extra consecutive classes needed:", extra)
```

8. Test Cases

Test Input	Expected / Checked Result
36 attended / 50 conducted	72.0%; 3 classes
40 / 50	80.0%; 0 classes
20 / 40	50.0%; 40 classes

Sample Output

```
Current attendance: 72.0 %\nExtra consecutive classes needed: 6
```

9. Conclusion

The program computes current attendance and uses a loop to find the minimum number of consecutive attended classes needed to meet the threshold.

13. ATM Transaction Sequence Simulation

1. Problem Statement

Simulate a sequence of deposits and withdrawals, maintain an account balance, reject invalid transactions, enforce a withdrawal limit, and display a transaction summary.

2. Objectives

• Process deposits and withdrawals in sequence.
• Update the balance correctly.
• Reject negative amounts and withdrawals above the limit or balance.
• Count successful and rejected transactions.
• Display the final balance.

3. Assumptions

• Opening balance is ■5,000.
• Each withdrawal is limited to ■2,000 and cannot exceed the available balance.
• Transactions are represented as (type, amount).

4. Algorithm

1. Start with the opening balance.
2. Read each transaction in order.
3. Reject non-positive amounts.
4. For a deposit, add the amount to the balance.
5. For a withdrawal, check the per-transaction limit and available balance.
6. Count successful or rejected transactions.
7. Display the final balance and counts.

5. Pseudocode

```
START
balance = 5000
FOR each transaction (type, amount)
IF amount <= 0: reject
ELSE IF type is DEPOSIT: add amount
ELSE IF type is WITHDRAW: check limit and balance, then subtract or reject
Print final balance and summary
STOP
```

6. Flowchart

START → Set opening balance → Read transaction → Validate amount → Deposit or withdrawal? → Check withdrawal limit/balance → Update or reject → More transactions? → Display summary → STOP

7. Python Source Code

```
balance = 5000
transactions = [("deposit", 1000), ("withdraw", 1500),
                ("withdraw", 2500), ("withdraw", 600)]
success = 0
rejected = 0
limit = 2000

for kind, amount in transactions:
    if amount <= 0:
        print("Rejected: invalid amount")
        rejected += 1
    elif kind == "deposit":
        balance += amount
        success += 1
        print("Deposit successful")
    elif kind == "withdraw":
        if amount > limit or amount > balance:
            print("Withdrawal rejected")
            rejected += 1
        else:
            balance -= amount
            success += 1
            print("Withdrawal successful")
    else:
        print("Rejected: invalid transaction type")
        rejected += 1
```

```
print("Final balance: Rs.", balance)
print("Successful transactions:", success)
print("Rejected transactions:", rejected)
```

8. Test Cases

Test Input	Expected / Checked Result
deposit 1000	successful; balance becomes 6000
withdraw 1500	successful; balance becomes 4500
withdraw 2500	rejected; exceeds limit

Sample Output

```
Deposit successful\nWithdrawal successful\nWithdrawal rejected\nWithdrawal successful\nFinal balance: Rs. 3900\nSuccessful
```

9. Conclusion

The simulation updates an account balance transaction by transaction and applies basic validation and withdrawal rules.

14. Warehouse Inventory Reconciliation

1. Problem Statement

Compare opening stock, received quantities, dispatched quantities, and recorded closing stock for multiple products. Identify discrepancies and calculate replenishment needs.

2. Objectives

• Calculate expected closing stock.
• Compare expected stock with recorded stock.
• Flag discrepancies.
• Calculate the quantity required to reach a target stock level.

3. Assumptions

• All quantities are non-negative integers.
• Expected closing stock = opening stock + received – dispatched.
• Target stock level is 30 units for each product.

4. Algorithm

1. Start.
2. Store each product's opening, received, dispatched, and recorded closing quantities.
3. Calculate expected closing stock.
4. Compare expected and recorded quantities.
5. Calculate shortage to target stock if expected stock is below target.
6. Display reconciliation results.

5. Pseudocode

```
START
FOR each product
expected = opening + received - dispatched
difference = recorded closing - expected
IF difference != 0, report discrepancy
replenishment = max(0, target - expected)
Display results
STOP
```

6. Flowchart

START → Read product stock data → Calculate expected closing stock → Compare with recorded closing → Discrepancy? → Calculate replenishment → Display report → STOP

7. Python Source Code

```
products = [
    ("Notebook", 20, 15, 10, 25),
    ("Pen", 40, 10, 25, 20),
    ("File", 18, 5, 8, 15)
]
target = 30

for name, opening, received, dispatched, recorded in products:
    expected = opening + received - dispatched
    difference = recorded - expected
    replenishment = max(0, target - expected)
    print(name, "Expected:", expected,
          "Recorded:", recorded, "Difference:", difference,
          "Replenish:", replenishment)
    if difference != 0:
        print("  Discrepancy detected")
```

8. Test Cases

Test Input	Expected / Checked Result
Notebook: 20+15–10	25; No discrepancy if recorded=25
Pen: 40+10–25	25; Difference –5 if recorded=20
File: 18+5–8	15; Replenish 15

Sample Output

```
Notebook Expected: 25 Recorded: 25 Difference: 0 Replenish: 5\nPen Expected: 25 Recorded: 20 Difference: -5 Replenish: 5\n
```

9. Conclusion

The program reconciles stock records, identifies mismatches, and calculates replenishment quantities from a target stock level.

15. Smart Parking Fee Management

1. Problem Statement

Calculate parking charges from entry and exit times, vehicle category, overnight parking, daily limits, and penalty conditions while validating time inputs.

2. Objectives

• Validate entry and exit times.
• Calculate parking duration, including overnight parking.
• Apply vehicle-specific hourly rates.
• Apply the daily charge cap and a penalty for invalid duration.

3. Assumptions

• Times are given as minutes after midnight.
• If exit time is earlier than entry time, the vehicle exited the next day.
• Car rate is ■30 per started hour; bike rate is ■15 per started hour.
• Maximum daily charge is ■300 for a car and ■150 for a bike.

4. Algorithm

1. Start.
2. Read vehicle type and entry/exit times.
3. Validate times from 0 to 1439.
4. Calculate duration; if exit is earlier, add 1440 minutes.
5. Round duration up to started hours.
6. Multiply by the vehicle rate and apply the daily cap.
7. Display duration and fee.

5. Pseudocode

```
START
Read vehicle, entry, exit
Validate 0 <= time < 1440
IF exit < entry: duration = exit + 1440 - entry ELSE duration = exit - entry
hours = ceiling(duration / 60)
fee = min(hours * rate, daily cap)
Display duration and fee
STOP
```

6. Flowchart

START → Enter vehicle and times → Validate input → Compute duration (handle overnight) → Calculate started hours → Apply rate and cap → Display fee → STOP

7. Python Source Code

```
import math

vehicle = "car"
entry = 22 * 60 + 30   # 10:30 PM
exit_time = 1 * 60 + 15  # 1:15 AM

if not (0 <= entry < 1440 and 0 <= exit_time < 1440):
    print("Invalid time")
else:
    duration = exit_time - entry
    if duration < 0:
        duration += 1440
    hours = math.ceil(duration / 60)

    if vehicle == "car":
        rate, cap = 30, 300
    elif vehicle == "bike":
        rate, cap = 15, 150
    else:
        rate, cap = 0, 0

    if rate == 0:
        print("Invalid vehicle type")
    else:
        fee = min(hours * rate, cap)
        print("Duration:", duration, "minutes")
```

```
print("Charged hours:", hours)
print("Parking fee: Rs.", fee)
```

8. Test Cases

Test Input	Expected / Checked Result
Car, 10:30 PM to 1:15 AM	165 minutes; 3 hours; ■90
Bike, 9:00 AM to 11:00 AM	120 minutes; 2 hours; ■30
Time = 1500	Invalid; Invalid time

Sample Output

```
Duration: 165 minutes\nCharged hours: 3\nParking fee: Rs. 90
```

9. Conclusion

The program handles overnight parking, validates time values, rounds up partial hours, and applies vehicle-specific rates and caps.

16. Multi-Day Water Tank Simulation

1. Problem Statement

Simulate a tank's water level over multiple intervals using changing inflow and outflow. Detect overflow, shortages, and the first interval where a target level is reached.

2. Objectives

• Update the tank level for each interval.
• Detect possible overflow and shortage.
• Keep the simulated level within the tank capacity.
• Find the first interval reaching a target level.

3. Assumptions

• Tank capacity is 1000 litres.
• Initial level is 500 litres.
• Inflow and outflow values are in litres per interval.
• Overflow is reported and level is capped; shortage is reported and level is set to zero.

4. Algorithm

1. Start with initial tank level.
2. For each interval, calculate level = current + inflow – outflow.
3. Report overflow if level exceeds capacity and cap it at capacity.
4. Report shortage if level is below zero and set it to zero.
5. Record the first interval where level reaches the target.
6. Display the level for each interval.

5. Pseudocode

```
START
level = initial level
FOR each interval
level = level + inflow - outflow
IF level > capacity: report overflow; level = capacity
IF level < 0: report shortage; level = 0
IF target not yet reached and level >= target: record interval
Display levels
STOP
```

6. Flowchart

START → Set capacity and initial level → Read interval inflow/outflow → Update level → Overflow or shortage? → Correct level bounds → Target reached first time? → Record → Display levels → STOP

7. Python Source Code

```
capacity = 1000
level = 500
target = 800
changes = [(300, 100), (250, 80), (50, 200), (100, 50)]
first_target_interval = None

for day, (inflow, outflow) in enumerate(changes, start=1):
    level = level + inflow - outflow
    if level > capacity:
        print("Day", day, ": overflow")
        level = capacity
    if level < 0:
        print("Day", day, ": water shortage")
        level = 0
    if level >= target and first_target_interval is None:
        first_target_interval = day
    print("Day", day, "level:", level, "litres")

print("First interval reaching target:", first_target_interval)
```

8. Test Cases

Test Input	Expected / Checked Result
Initial 500; inflow 300; outflow 100	700 L; No target yet

Test Input	Expected / Checked Result
Next inflow 250; outflow 80	870 L; Target reached on interval 2
Outflow exceeds available water	0 L; Shortage reported

Sample Output

```
Day 1 level: 700 litres\nDay 2 level: 870 litres\nDay 3 level: 720 litres\nDay 4 level: 770 litres\nFirst interval reachin
```

9. Conclusion

The simulation updates water levels over time and identifies boundary violations and the first interval reaching a target.

17. Student Performance Trend Analysis

1. Problem Statement

Compare a student's marks across multiple examinations, calculate improvements or declines, identify weak subjects, and classify overall performance.

2. Objectives

• Store marks for multiple examinations.
• Calculate change in each subject.
• Identify subjects with low marks.
• Calculate average performance and assign a classification.

3. Assumptions

• Marks are out of 100.
• Marks below 40 are considered weak.
• Average of 75 or more is Excellent, 60 or more is Good, otherwise Needs Improvement.

4. Algorithm

1. Store subject names and marks from two exams.
2. For each subject calculate the difference between new and old marks.
3. Flag subjects with current marks below 40.
4. Calculate the current average.
5. Classify performance using the given thresholds.
6. Display the trend report.

5. Pseudocode

```
START
Store old and current marks
FOR each subject: change = current - old
IF current < 40, mark subject as weak
average = sum(current marks) / number of subjects
Classify average
Display changes, weak subjects and classification
STOP
```

6. Flowchart

START → Enter marks from two exams → Calculate subject-wise changes → Identify marks below 40 → Calculate current average → Classify performance → Display report → STOP

7. Python Source Code

```
subjects = ["Maths", "Physics", "Python"]
previous = [55, 38, 62]
current = [68, 42, 58]

for i in range(len(subjects)):
    change = current[i] - previous[i]
    print(subjects[i], "Change:", change,
          "Weak:", current[i] < 40)

average = sum(current) / len(current)
if average >= 75:
    category = "Excellent"
elif average >= 60:
    category = "Good"
else:
    category = "Needs Improvement"

print("Current average:", round(average, 2))
print("Classification:", category)
```

8. Test Cases

Test Input	Expected / Checked Result
55→68, 38→42, 62→58	Changes +13, +4, –4; Average 56.0; Needs Improvement
All current marks 80	Average 80; Excellent

Test Input	Expected / Checked Result
Current mark 35 in one subject	Below 40; Weak subject flagged

Sample Output

```
Maths Change: 13 Weak: False\nPhysics Change: 4 Weak: False\nPython Change: -4 Weak: False\nCurrent average: 56.0\nClassif
```

9. Conclusion

The program compares examination results, identifies weak subjects, calculates averages, and classifies performance using clear rules.

18. Fuel Consumption and Route Comparison

1. Problem Statement

Compare vehicles for a journey by calculating fuel needed and journey cost from distance, mileage, fuel price, and reserve requirements. Select the lowest-cost feasible vehicle.

2. Objectives

• Calculate fuel required for each vehicle.
• Include a required reserve quantity.
• Calculate fuel cost.
• Reject vehicles that cannot carry enough fuel.
• Select the lowest-cost feasible option.

3. Assumptions

• Journey distance is 300 km.
• Fuel price is ■105 per litre.
• A reserve of 2 litres must remain available.
• Each vehicle has a stated tank capacity and mileage.

4. Algorithm

1. Store each vehicle's mileage and tank capacity.
2. Calculate journey fuel = distance / mileage.
3. Calculate required fuel including reserve.
4. Check whether required fuel fits within capacity.
5. Calculate journey cost for feasible vehicles.
6. Select and display the feasible vehicle with the lowest cost.

5. Pseudocode

```
START
Set distance, fuel price, reserve
FOR each vehicle
journey_fuel = distance / mileage
required = journey_fuel + reserve
IF required <= tank capacity: calculate cost
Keep the feasible vehicle with minimum cost
Display result
STOP
```

6. Flowchart

START → Enter route and vehicle data → Calculate fuel needed → Add reserve → Fits tank capacity? → YES: calculate cost / NO: reject → Compare feasible costs → Display cheapest option → STOP

7. Python Source Code

```
distance = 300
fuel_price = 105
reserve = 2
vehicles = [
    ("Car A", 15, 30),
    ("Car B", 20, 18),
    ("Car C", 12, 35)
]
best = None

for name, mileage, capacity in vehicles:
    journey_fuel = distance / mileage
    required = journey_fuel + reserve
    if required <= capacity:
        cost = journey_fuel * fuel_price
        print(name, "Fuel:", round(journey_fuel, 2),
              "Cost: Rs.", round(cost, 2))
        if best is None or cost < best[1]:
            best = (name, cost)
    else:
        print(name, "Not feasible: insufficient tank capacity")

if best:
    print("Lowest-cost feasible vehicle:", best[0])
    print("Estimated cost: Rs.", round(best[1], 2))
```

```
else:
    print("No feasible vehicle")
```

8. Test Cases

Test Input	Expected / Checked Result
Car A: 15 km/L, 30 L tank	20 L + 2 L reserve; Feasible; ■2100
Car B: 20 km/L, 18 L tank	15 L + 2 L reserve; Feasible; ■1575
Car C: 12 km/L, 35 L tank	25 L + 2 L reserve; Feasible; ■2625

Sample Output

```
Car A Fuel: 20.0 Cost: Rs. 2100.0\nCar B Fuel: 15.0 Cost: Rs. 1575.0\nCar C Fuel: 25.0 Cost: Rs. 2625.0\nLowest-cost feasi
```

9. Conclusion

The program compares fuel requirements and costs while considering tank capacity and reserve fuel, then selects the cheapest feasible vehicle.

19. Multi-Product Shopping Budget Constraint

1. Problem Statement

Evaluate a shopping basket with product quantities, apply discount conditions, and determine which purchases must be reduced to stay within a fixed budget.

2. Objectives

• Calculate each product subtotal.
• Apply a discount when the basket reaches a threshold.
• Calculate the final total.
• Compare the total with the budget.
• Suggest reductions if the basket exceeds the budget.

3. Assumptions

• Budget is ■2,000.
• A 10% discount applies when the pre-discount total is at least ■1,500.
• If over budget, reduce quantity of the most expensive item one unit at a time.

4. Algorithm

1. Store product name, unit price, and quantity.
2. Calculate the basket subtotal.
3. Apply the threshold discount if eligible.
4. While total exceeds budget, reduce one unit from the most expensive item with quantity above zero and recalculate.
5. Display final quantities, discount, and total.

5. Pseudocode

```
START
Store products, prices and quantities
Calculate subtotal
IF subtotal >= 1500: discount = 10% ELSE discount = 0
WHILE final total > budget: reduce one unit of highest-priced available product; recalculate
Display final basket and total
STOP
```

6. Flowchart

START → Enter basket and budget → Calculate subtotal → Apply discount rule → Total within budget? → NO: reduce highest-priced item quantity and recalculate → YES: display final basket → STOP

7. Python Source Code

```
budget = 2000
discount_rate = 0.10
items = [
    ["Headphones", 900, 1],
    ["Notebook", 100, 5],
    ["Bottle", 250, 2]
]

def calculate_total():
    subtotal = sum(price * qty for name, price, qty in items)
    discount = subtotal * discount_rate if subtotal >= 1500 else 0
    return subtotal, discount, subtotal - discount

subtotal, discount, total = calculate_total()
while total > budget:
    candidates = [item for item in items if item[2] > 0]
    if not candidates:
        break
    highest = max(candidates, key=lambda item: item[1])
    highest[2] -= 1
    subtotal, discount, total = calculate_total()

print("Final basket:")
for name, price, qty in items:
    print(name, "Quantity:", qty)
print("Subtotal:", subtotal)
print("Discount:", discount)
print("Final total:", total)
```

8. Test Cases

Test Input	Expected / Checked Result
Subtotal ■1,900	10% discount; Final total ■1,710; within budget
Subtotal ■2,500	10% discount; Reduce quantities until ≤ ■2,000
All quantities zero	■0; Within budget

Sample Output

```
Final basket:\nHeadphones Quantity: 0\nNotebook Quantity: 5\nBottle Quantity: 2\nSubtotal: 1000\nDiscount: 0\nFinal total:
```

9. Conclusion

The program calculates discounts and adjusts quantities to bring the shopping basket within the specified budget.

20. University Admission Decision Engine

1. Problem Statement

Evaluate university applicants using subject-wise eligibility, aggregate marks, course-specific minimum thresholds, and tie-breaking rules. Display eligible applicants in rank order and list ineligible applicants.

2. Objectives

• Check minimum marks in each subject.
• Calculate aggregate percentage.
• Apply course-specific thresholds.
• Separate eligible and ineligible applicants.
• Rank eligible applicants using tie-breaking rules.

3. Assumptions

• Subjects are Mathematics, Physics, and Computer Science.
• Minimum mark in each subject is 50.
• Thresholds: CSE 70%, ECE 65%, EEE 60%.
• Ties are resolved by higher Mathematics, then higher Computer Science, then alphabetical name.

4. Algorithm

1. Store applicant records and course thresholds.
2. Calculate each applicant's aggregate.
3. Check subject minimums and course threshold.
4. Separate eligible and ineligible applicants.
5. Sort eligible applicants by aggregate, Mathematics, Computer Science, and name.
6. Display ranking and ineligible list.

5. Pseudocode

```
START
Set subject minimum = 50 and course thresholds
FOR each applicant: calculate aggregate
Check subject marks and course threshold
Store in eligible or ineligible list
Sort eligible by aggregate descending, Maths descending, Computer Science descending, name ascending
Display ranking and ineligible list
STOP
```

6. Flowchart

START → Store applicants → Set thresholds → Calculate aggregate → Check subject minimums and course threshold → Eligible? → YES: eligible list / NO: ineligible list → Rank eligible applicants using tie-breakers → Display results → STOP

7. Python Source Code

```
thresholds = {"CSE": 70, "ECE": 65, "EEE": 60}
minimum_mark = 50
applicants = [
    ("Anil", "CSE", 82, 76, 88),
    ("Bhavya", "ECE", 70, 68, 72),
    ("Charan", "CSE", 74, 70, 76),
    ("Divya", "EEE", 65, 60, 70),
    ("Esha", "CSE", 69, 72, 75),
    ("Farhan", "ECE", 70, 68, 72)
]
eligible = []
ineligible = []

for name, course, maths, physics, computer in applicants:
    average = (maths + physics + computer) / 3
    valid_subjects = all(mark >= minimum_mark
                         for mark in (maths, physics, computer))
    if valid_subjects and average >= thresholds[course]:
        eligible.append((name, course, maths, physics, computer, average))
    else:
        ineligible.append((name, course, round(average, 2)))

eligible.sort(key=lambda a: (-a[5], -a[2], -a[4], a[0]))

print("UNIVERSITY ADMISSION RANKING")
for rank, applicant in enumerate(eligible, start=1):
```

```
    name, course, maths, physics, computer, average = applicant
    print(rank, name, course, round(average, 2), "%")

print("INELIGIBLE APPLICANTS")
for applicant in ineligible:
    print(applicant[0], applicant[1], applicant[2], "%")
```

8. Test Cases

Test Input	Expected / Checked Result
Anil, CSE, 82/76/88	Average 82%; Eligible; Rank 1
Esha, CSE, 69/72/75	Average 72%; Ineligible: Maths below 70 aggregate threshold not met
Bhavya and Farhan, ECE, 70/68/72	Average 70% each; Tie resolved alphabetically

Sample Output

```
UNIVERSITY ADMISSION RANKING\n1 Anil CSE 82.0 %\n2 Charan CSE 73.33 %\n3 Bhavya ECE 70.0 %\n4 Farhan ECE 70.0 %\n5 Divya E
```

9. Conclusion

The program evaluates admission eligibility, applies course thresholds, separates applicants, and ranks eligible candidates using the specified tie-breaking rules.